



INTEGRANTES: Andrade Hugo, Dominguez Daniel, Gualle Abisag, Guanga Sebastian, Guanotoa Karla

CARRERA: Software.

PARALELO: “A”

ASIGNATURA: Algoritmos y Lógica de Programación

NIVEL: 1

FECHA: 19/05/2026

DOCENTE: Ing. José Caiza

TEMA: Operaciones Avanzadas e Integridad de Datos

LINK DE GIT HUB: <https://github.com/Abisag-Lisenia/Tarea-6-Operaciones-Avanzadas-e-Integridad-de-Datos/tree/main>

RESUMEN.

El presente informe académico expone el desarrollo práctico y el análisis de algoritmos orientados a la optimización de procesos de búsqueda y a la seguridad en el manejo de estructuras de datos estáticas en el lenguaje de programación Java. El trabajo se divide en dos secciones principales orientadas a la integridad del software: en primer lugar, se implementó un algoritmo de búsqueda lineal utilizando arreglos paralelos para la gestión de un inventario de productos y sus respectivos precios, optimizando el tiempo de respuesta mediante la interrupción controlada del ciclo con la instrucción `break` al hallar la coincidencia. En segundo lugar, se abordó la prevención de la vulnerabilidad de desbordamiento de búfer (*Buffer Overflow*) mediante la correcta delimitación de las condiciones de parada en bucles iterativos basadas en la propiedad `.length` del arreglo. Las pruebas de ejecución realizadas en el entorno de desarrollo permitieron comprobar la efectividad de los controles lógicos implementados y analizar la excepción `ArrayIndexOutOfBoundsException` como resultado de accesos fuera de los límites de memoria permitidos. Los resultados consolidan la importancia de integrar la eficiencia algorítmica con las buenas prácticas de diseño seguro para garantizar la robustez y disponibilidad de las aplicaciones.



ÍNDICE.

INTRODUCCIÓN.	3
OBJETIVO.	3
OBJETIVOS ESPECÍFICOS.	3
DESARROLLO DE ACTIVIDADES.	4
CONCLUSIONES.	19
RECOMENDACIONES.	19
ANEXOS.	19

INTRODUCCIÓN.

En el ámbito del desarrollo de software y la ingeniería informática, el manejo eficiente y seguro de las estructuras de datos fundamentales constituye la base para construir aplicaciones robustas, escalables y confiables. Los arreglos (o *arrays*), como estructuras lineales homogéneas, representan una de las herramientas más utilizadas para el almacenamiento y la organización de información en memoria ram debido a su acceso indexado y su predictibilidad operativa. Sin embargo, el uso de estas estructuras estáticas conlleva la responsabilidad de implementar algoritmos óptimos de manipulación y, fundamentalmente, estrictos mecanismos de control que aseguren la validez de los datos y el comportamiento del sistema.

El presente informe académico documenta el diseño, desarrollo y validación de dos componentes prácticos esenciales en la formación de la lógica de programación: las operaciones de búsqueda avanzada y la prevención de vulnerabilidades de memoria. En primera instancia, se aborda el algoritmo de búsqueda lineal aplicado sobre arreglos paralelos, una técnica que permite asociar diferentes atributos de una entidad —como productos y sus respectivos costos— compartiendo un mismo índice vinculante. A través de esta práctica, se examina cómo controlar el flujo de iteración mediante saltos condicionales (*break*) para mejorar el rendimiento temporal del algoritmo cuando el elemento objetivo es localizado.

En segunda instancia, el enfoque se desplaza hacia la seguridad en el desarrollo de código a través del análisis de la prevención del desbordamiento de búfer (*Buffer Overflow*). Este fenómeno, considerado una de las vulnerabilidades más comunes y críticas en la informática, ocurre cuando un programa intenta escribir o leer datos fuera de los límites asignados a un bloque de memoria. Mediante el uso del lenguaje de programación Java, se demuestra la importancia de validar rigurosamente las cotas superiores de los ciclos iterativos utilizando las propiedades nativas de la estructura (*.length*), evitando así que el sistema invoque excepciones críticas como *ArrayIndexOutOfBoundsException* que comprometan la disponibilidad del software. A través de este trabajo, el equipo consolida la importancia de fusionar la eficiencia algorítmica con las buenas prácticas de integridad de datos.

OBJETIVO.

Desarrollar e implementar programas estructurados en el lenguaje de programación Java que apliquen operaciones avanzadas de búsqueda y mecanismos de control lógico, con la finalidad de garantizar la integridad de los datos y prevenir vulnerabilidades críticas como el desbordamiento de memoria (*Buffer Overflow*) durante la manipulación de arreglos.

OBJETIVOS ESPECÍFICOS.

- Diseñar e implementar un algoritmo de búsqueda lineal mediante el uso de arreglos paralelos para asociar eficientemente las propiedades de un conjunto de elementos (productos y precios) y optimizar el tiempo de ejecución empleando estructuras de control de salto indexado (*break*).
- Mitigar los riesgos asociados al desbordamiento de búfer y errores en tiempo de ejecución analizando exhaustivamente los límites lógicos de los ciclos iterativos y aplicando la validación de cotas superiores basadas en la propiedad *.length* de las estructuras de datos bidimensionales o unidimensionales.
- Validar la robustez, el comportamiento y el control de excepciones de los códigos fuentes desarrollados mediante el diseño de casos de prueba específicos que incluyan tanto ingresos válidos como escenarios propensos a errores operacionales.

DESARROLLO DE ACTIVIDADES.

1) Enunciado del Ejercicio 1.

Búsqueda Lineal de Productos.

Desarrollar un programa en C++ o Java que permita buscar un producto dentro de un arreglo utilizando el algoritmo de búsqueda lineal.

Indicaciones:

Crear un arreglo con 5 nombres de productos.

Crear otro arreglo paralelo con sus precios.

Solicitar al usuario el nombre del producto a buscar.

Recorrer el arreglo usando búsqueda lineal.

Mostrar:

Nombre del producto.

Precio correspondiente.

Utilizar break cuando se encuentre el producto.

Mostrar un mensaje si el producto no existe.

2) Análisis del problema.

Para resolver este ejercicio mediante una **búsqueda lineal**, necesitamos recorrer secuencialmente una estructura de datos (en este caso, un arreglo) hasta encontrar el valor deseado o hasta llegar al final del arreglo.

Variables

- **Variables de entrada:**

- producto_buscado (Texto / String)

- **Variables de salida:**

- Nombre del producto encontrado (Texto).
 - Precio del producto encontrado (Decimal / Float).
 - Mensaje de error si no hay coincidencias (Texto).

Proceso a realizar

1. Inicializar dos arreglos paralelos de tamaño 5: uno para los nombres de los productos y otro para sus precios respectivos (el índice 0 del primer arreglo corresponde al índice 0 del segundo).
2. Leer desde el teclado el nombre del producto que se desea buscar.
3. Iniciar un bucle que itere desde el índice 0 hasta el 4.
4. En cada iteración, comparar el elemento actual del arreglo de nombres con el producto_buscado.
5. Si los nombres coinciden, imprimir el nombre y el precio, cambiar el estado de la variable booleana a verdadero, y ejecutar un break para detener el bucle inmediatamente.

Al finalizar el bucle, evaluar la variable booleana. Si sigue siendo falsa, imprimir un mensaje indicando que el producto no existe.

3) Algoritmo.

1. Inicio

2. Definir arreglo nombres = {"Microcontrolador", "Sensor", "Resistencia", "Motor", "Pantalla"}
3. Definir arreglo precios = {15.50, 4.25, 0.10, 8.75, 12.00}
4. Definir variable booleana encontrado = Falso
5. Mostrar mensaje: "Ingrese el producto a buscar:"
6. Leer producto_buscado
7. Para i desde 0 hasta 4, hacer:
 - Si nombres[i] es exactamente igual a producto_buscado, entonces:
 - Imprimir nombres[i] y precios[i]
 - encontrado = Verdadero
 - Romper el ciclo (break)
8. Si encontrado es igual a Falso, entonces:
 - Imprimir "El producto no existe."
9. **Fin**

4) Pseudocódigo.

Algoritmo Busqueda_Productos

// 1. Declarar los tipos de variables y arreglos

Definir producto Como Cadena

Definir precio Como Real

Definir buscar Como Cadena

Definir encontrado Como Logico

Definir i Como Entero

// 2. Dimensionar los arreglos (Se indica el tamaño)

Dimension producto[5]

Dimension precio[5]

// 3. Asignar los valores a los arreglos (Índices del 1 al 5)

producto[1] <- "Leche"

producto[2] <- "Pan"

producto[3] <- "Yogurt"

```
producto[4] <- "Cereal"
```

```
producto[5] <- "Sandia"
```

```
precio[1] <- 1.0
```

```
precio[2] <- 0.25
```

```
precio[3] <- 0.75
```

```
precio[4] <- 1.5
```

```
precio[5] <- 2.75
```

```
// Inicializar la bandera
```

```
encontrado <- Falso
```

```
// 4. Solicitar datos al usuario
```

```
Escribir "Ingrese el producto que desea buscar: "
```

```
Leer buscar
```

```
// 5. Ciclo de búsqueda lineal
```

```
Para i <- 1 Hasta 5 Con Paso 1 Hacer
```

```
    Si producto[i] = buscar Entonces
```

```
        Escribir "Producto encontrado!"
```

```
        Escribir "Producto: ", producto[i]
```

```
        Escribir "Precio: ", precio[i]
```

```
        encontrado <- Verdadero
```

```
        i <- 5 // Reemplaza al 'break' para forzar la salida del ciclo Para
```

```
    FinSi
```

```
FinPara
```

```
// 6. Mensaje en caso de no éxito
```

```
Si NO encontrado Entonces
```

```
    Escribir "No se dispone del producto ingresado"
```

```
FinSi
```

FinAlgoritmo

5) Diagrama de Flujo.

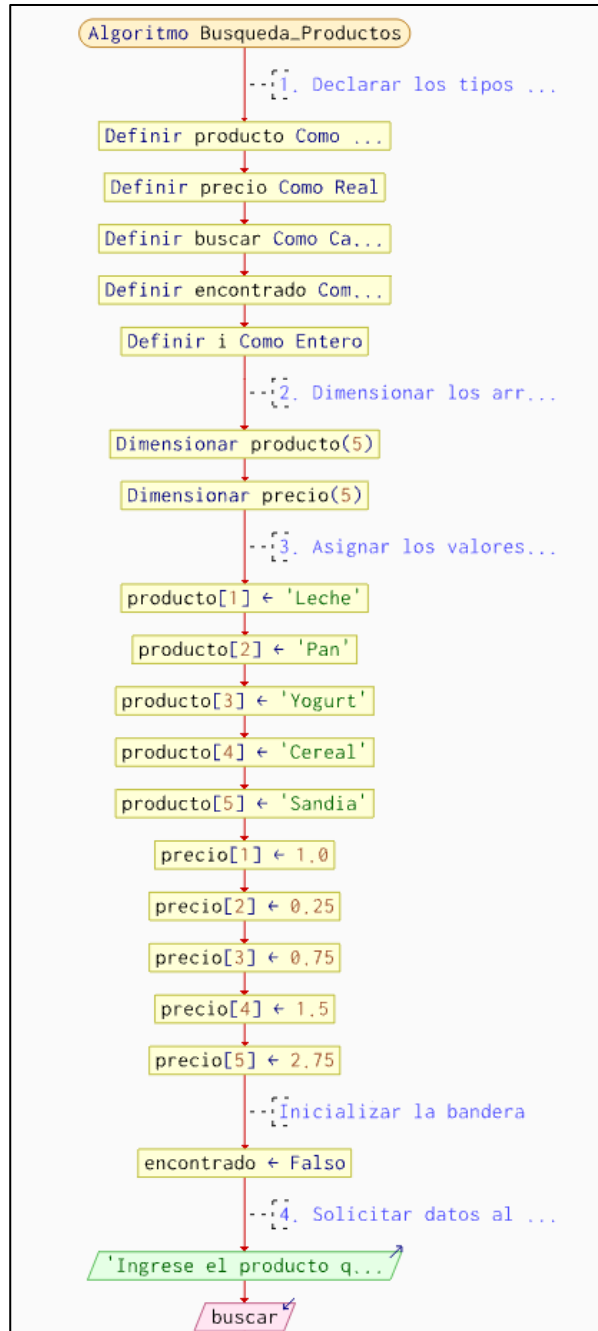


Ilustración 0.1: Diagrama de flujo

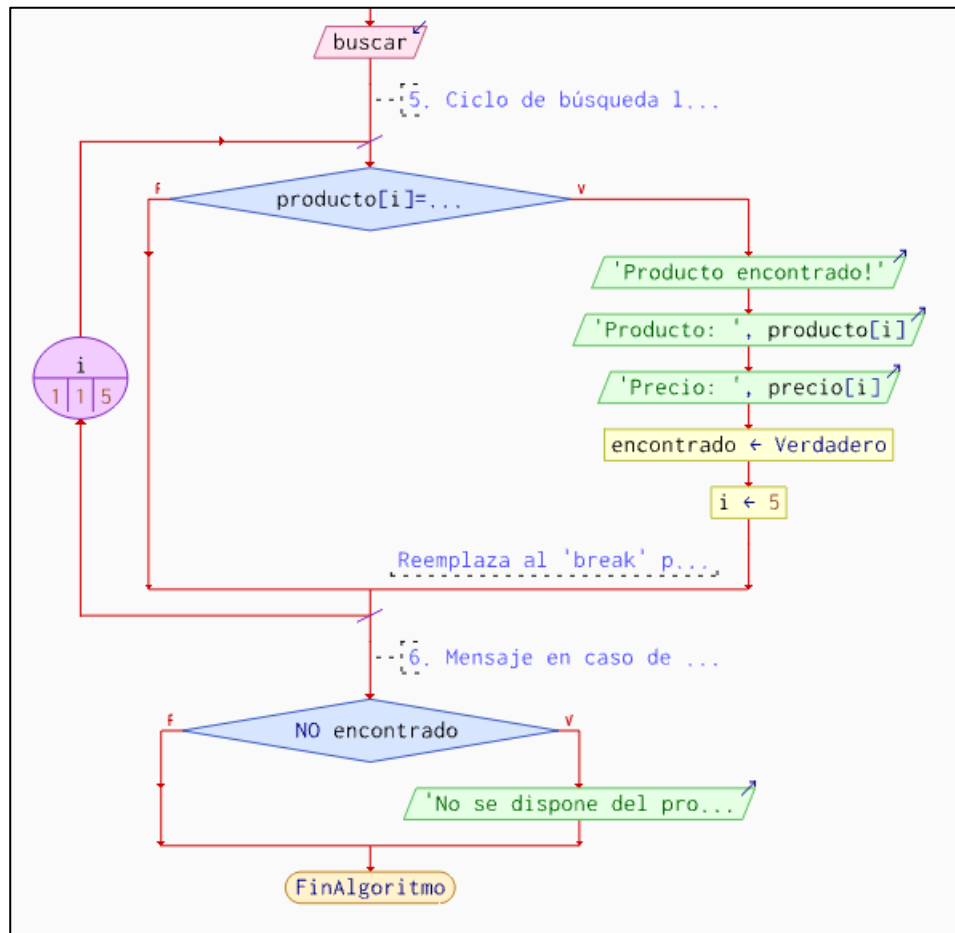


Ilustración 0.2: Diagrama de flujo

6) **Código en C++.**
Ejercicio 1 en C++
`#include<iostream>`
`using namespace std;`

`int main(){`

`//Declaracion de productos y precios`

`string producto[5]={"Leche", "Pan", "Yogurt", "Cereal", "Sandia"};`

`float precio[5]={1,0.25,0.75,1.5,2.75};`

`//Declaracion de variables a buscar`

`string buscar;`

`bool encontrado = false;`



```
//Pedir el producto al usuario
cout << "Ingrese el producto que desea buscar: ";
cin >> buscar;

//Busqueda lineal con un for
for(int i=0; i<5; i++){

    if(producto[i]==buscar){
        cout << "Producto encontrado!\n";
        cout << "Producto: " << producto[i]
        << "\nPrecio: " << precio[i] << endl;

        encontrado=true;
        break;
    }
}

//En caso de que no se encuentre el producto
if(!encontrado){
    cout << "No se dispone del producto ingresado";
}

return 0;

}
```

7) Código en Java.
Ejercicio 1 en Java

```
import java.util.Scanner;

public class BuscarProducto {
    public static void main(String[] args) {
```



```
// Declaración de un arreglo de productos
String[] producto = {"Leche", "Pan", "Yogurt", "Cereal", "Sandia"};

// Declaración de un arreglo de precios (correspondiente a cada producto)
float[] precio = {1f, 0.25f, 0.75f, 1.5f, 2.75f};

// Objeto Scanner para leer datos desde teclado
Scanner sc = new Scanner(System.in);

// Solicitar al usuario el producto que desea buscar
System.out.print("Ingrese el producto que desea buscar: ");
String buscar = sc.nextLine();

// Variable booleana para indicar si se encontró el producto
boolean encontrado = false;

// Recorrer el arreglo de productos
for (int i = 0; i < producto.length; i++) {
    // Comparar el producto ingresado con el arreglo (ignora
    // mayúsculas/minúsculas)
    if (producto[i].equalsIgnoreCase(buscar)) {
        System.out.println("Producto encontrado!");
        System.out.println("Producto: " + producto[i]);
        System.out.println("Precio: " + precio[i]);
        encontrado = true; // marcar que sí se encontró
        break; // salir del ciclo porque ya se encontró
    }
}

// Si no se encontró el producto, mostrar mensaje
if (!encontrado) {
    System.out.println("No se dispone del producto ingresado");
}

// Cerrar el objeto Scanner
```

```
sc.close();
```

```
}
```

8) Pruebas de solución.

```
int main(){
    //Declaracion de productos y precios
    string producto[5]={"Leche", "Pan", "Yogurt", "Cereal", "Sandia"};
    float precio[5]={1,0.25,0.75,1.5,2.75};

    //Declaracion de variables a buscar
    string buscar;
    bool encontrado = false;

    //Pedir el producto al usuario
    cout << "Ingrese el producto que desea buscar: ";
    cin >> buscar;

    //Busqueda lineal con un for
    for(int i=0; i<5; i++){
        if(producto[i]==buscar){
            cout << "Producto encontrado!\n";
            cout << "Producto: " << producto[i]
            << "\nPrecio: " << precio[i] << endl;

            encontrado=true;
            break;
        }
    }

    //En caso de que no se encuentre el producto
    if(!encontrado){
        cout << "No se dispone del producto ingresado";
    }
    return 0;
}
```

Ilustración 0.3: Código de C++ en Visual Studio Core

```
PS C:\Users\LENOVO LOQ\Documents\Visual Studio Code\C++> cd "c:\Users\LENOVO LOQ\Documents\Visual Studio Code\C++\" ; if ($?) { g++ Tarea6.cpp -o Tarea6 } ; if ($?) { .\Tarea6 }
Ingrese el producto que desea buscar: Leche
Producto encontrado!
Producto: Leche
Precio: 1
```

Ilustración 0.4: Prueba de solución 1 C++

```
PS C:\Users\LENOVO LOQ\Documents\Visual Studio Code\C++> cd "c:\Users\LENOVO LOQ\Documents\Visual Studio Code\C++\" ; if ($?) { g++ Tarea6.cpp -o Tarea6 } ; if ($?) { .\Tarea6 }
Ingrese el producto que desea buscar: Pan
Producto encontrado!
Producto: Pan
Precio: 0.25
PS C:\Users\LENOVO LOQ\Documents\Visual Studio Code\C++> █
```

Ilustración 0.5: Prueba de solución 2 C++

```
PS C:\Users\LENOVO LOQ\Documents\Visual Studio Code\C++> cd "c:\Users\LENOVO LOQ\Documents\Visual Studio Code\C++\" ; if ($?) { g++ Tarea6.cpp -o Tarea6 } ; if ($?) { .\Tarea6 }
Ingrese el producto que desea buscar: Jamon
No se dispone del producto ingresado
PS C:\Users\LENOVO LOQ\Documents\Visual Studio Code\C++> █
```

Ilustración 0.6: Prueba de solución 3 C++

```
PS C:\Users\LENOVO LOQ\Documents\Visual Studio Code\C++> cd "c:\Users\LENOVO LOQ\Documents\Visual Studio Code\C++\" ; if ($?) { g++ Tarea6.cpp -o Tarea6 } ; if ($?) { .\Tarea6 }
Ingrese el producto que desea buscar: Sandia
Producto encontrado!
Producto: Sandia
Precio: 2.75
PS C:\Users\LENOVO LOQ\Documents\Visual Studio Code\C++> |
```

Ilustración 0.7: Prueba de solución 4 C++

```
J BuscarProducto.java > BuscarProducto > main(String[])
1 import java.util.Scanner;
2
3 public class BuscarProducto {
4     public static void main(String[] args) {
5         // Declaración de un arreglo de productos
6         String[] producto = {"Leche", "Pan", "Yogurt", "Cereal", "Sandia"};
7         // Declaración de un arreglo de precios (correspondiente a cada producto)
8         float[] precio = {1f, 0.25f, 0.75f, 1.5f, 2.75f};
9
10        // Objeto Scanner para leer datos desde teclado
11        Scanner sc = new Scanner(System.in);
12
13        // Solicitar al usuario el producto que desea buscar
14        System.out.print(s: "Ingrese el producto que desea buscar: ");
15        String buscar = sc.nextLine();
16
17        // Variable booleana para indicar si se encontró el producto
18        boolean encontrado = false;
19
20        // Recorrer el arreglo de productos
21        for (int i = 0; i < producto.length; i++) {
22            // Comparar el producto ingresado con el arreglo (ignora mayúsculas/minúsculas)
23            if (producto[i].equalsIgnoreCase(buscar)) {
24                System.out.println(x: "Producto encontrado!");
25                System.out.println("Producto: " + producto[i]);
26                System.out.println("Precio: " + precio[i]);
27                encontrado = true; // marcar que sí se encontró
28                break; // salir del ciclo porque ya se encontró
29            }
30        }
31
32        // Si no se encontró el producto, mostrar mensaje
33        if (!encontrado) {
34            System.out.println(x: "No se dispone del producto ingresado");
35        }
36
37        // Cerrar el objeto Scanner
38        sc.close();
39    }
40 }
```

Ilustración 0.8: Código de Java en Visual Studio Core

```
PS C:\Users\HP\Documents\GitHub\P00> c:.; cd 'c:\Users\HP\Documents\GitHub\P00';
onMessages' '-cp' 'C:\Users\HP\AppData\Roaming\Code\User\workspaceStorage\d16aa3
Ingrese el producto que desea buscar: Leche
Producto encontrado!
Producto: Leche
Precio: 1.0
```

Ilustración 0.9: Prueba de solución 1 Java

```
S C:\Users\HP\Documents\GitHub\P00> c:.; cd 'c:\Users\HP\Documents\GitHub\P00';
onMessages' '-cp' 'C:\Users\HP\AppData\Roaming\Code\User\workspaceStorage\d16aa3
ngrese el producto que desea buscar: queso
o se dispone del producto ingresado
S C:\Users\HP\Documents\GitHub\P00> |
```

Ilustración 0.10: Prueba de solución 2 Java

```
PS C:\Users\HP\Documents\GitHub\P00> c++; cd 'c:\Users\HP\Documents\GitHub\P00';  
onMessages' '-cp' 'C:\Users\HP\AppData\Roaming\Code\User\workspaceStorage\d16aa;  
Ingrese el producto que desea buscar: Cereal  
Producto encontrado!  
Producto: Cereal  
Precio: 1.5  
PS C:\Users\HP\Documents\GitHub\P00> █
```

Ilustración 0.11: Prueba de solución 3 Java

1) Enunciado del Ejercicio 2.

Prevención de Buffer Overflow

Realizar un programa en C++ o Java que muestre los elementos de un arreglo sin provocar desbordamiento de memoria.

Indicaciones:

Crear un arreglo con 6 números enteros.

Utilizar un ciclo for para recorrer el arreglo.

Validar correctamente los límites del ciclo.

Mostrar cada posición y su valor.

Agregar un comentario explicando qué ocurriría si el ciclo supera el tamaño del arreglo.

2) Análisis del problema.

El problema central radica en el manejo correcto de la memoria. Los arreglos tienen un tamaño fijo, y sus índices comienzan en 0. Para un arreglo de 6 elementos, los índices válidos van del 0 al 5. Acceder a un índice igual o mayor a 6 es un error crítico. La validación del límite del ciclo es la principal técnica para prevenir este desbordamiento.

Variables

• **Variables de entrada:**

- No hay variables ingresadas por el usuario en este ejercicio. Los datos se definen estáticamente en el código.

• **Variables de salida:**

- Índice o posición de memoria (Entero).
- Valor almacenado en esa posición (Entero).

Proceso a realizar:

1. Declarar e inicializar un arreglo unidimensional de tamaño 6 con valores enteros.
2. Establecer un ciclo iterativo for que inicie en el índice 0.
3. Configurar la condición de parada del ciclo para que sea estrictamente menor al tamaño total del arreglo (límite inferior estricto a 6).
4. Dentro del ciclo, extraer e imprimir el valor del iterador actual (posición) junto con el valor del arreglo en esa posición.

3) Algoritmo.

1. Inicio

2. Definir un arreglo llamado numeros con 6 valores enteros: {10, 20, 30, 40, 50, 60}.
3. Definir el tamaño máximo permitido (límite = 6).
4. Para i empezando desde 0, y mientras i sea menor que 6, hacer:

- Imprimir la frase "Posición [" e i y "]" -> Valor: " y numeros[i].
- Incrementar i en 1.

5. Fin

4) **Pseudocódigo.** *Algoritmo Mostrar_Arreglo*

// 1. Declarar el arreglo y la variable del ciclo

Definir numeros Como Entero

Definir i Como Entero

// 2. Dimensionar el arreglo con tamaño 6

Dimension numeros[6]

// 3. Asignar los valores uno por uno (En PSeInt los índices van del 1 al 6)

numeros[1] <- 10

numeros[2] <- 20

numeros[3] <- 30

numeros[4] <- 40

numeros[5] <- 50

numeros[6] <- 60

// 4. Ciclo Para con los límites correctos para recorrer el arreglo

Para i <- 1 Hasta 6 Con Paso 1 Hacer

// Mostrar posicion y valor

Escribir "Posicion [" , i , "]" = " , numeros[i]

FinPara

// Si el ciclo supera el tamaño del arreglo (por ejemplo i <- 1 Hasta 7),

// PSeInt lanzará un error de "Índice fuera de rango" deteniendo la ejecución,

// ya que intentarías acceder a una posición que no ha sido reservada en la dimensión.

FinAlgoritmo

5) **Diagrama de Flujo.**

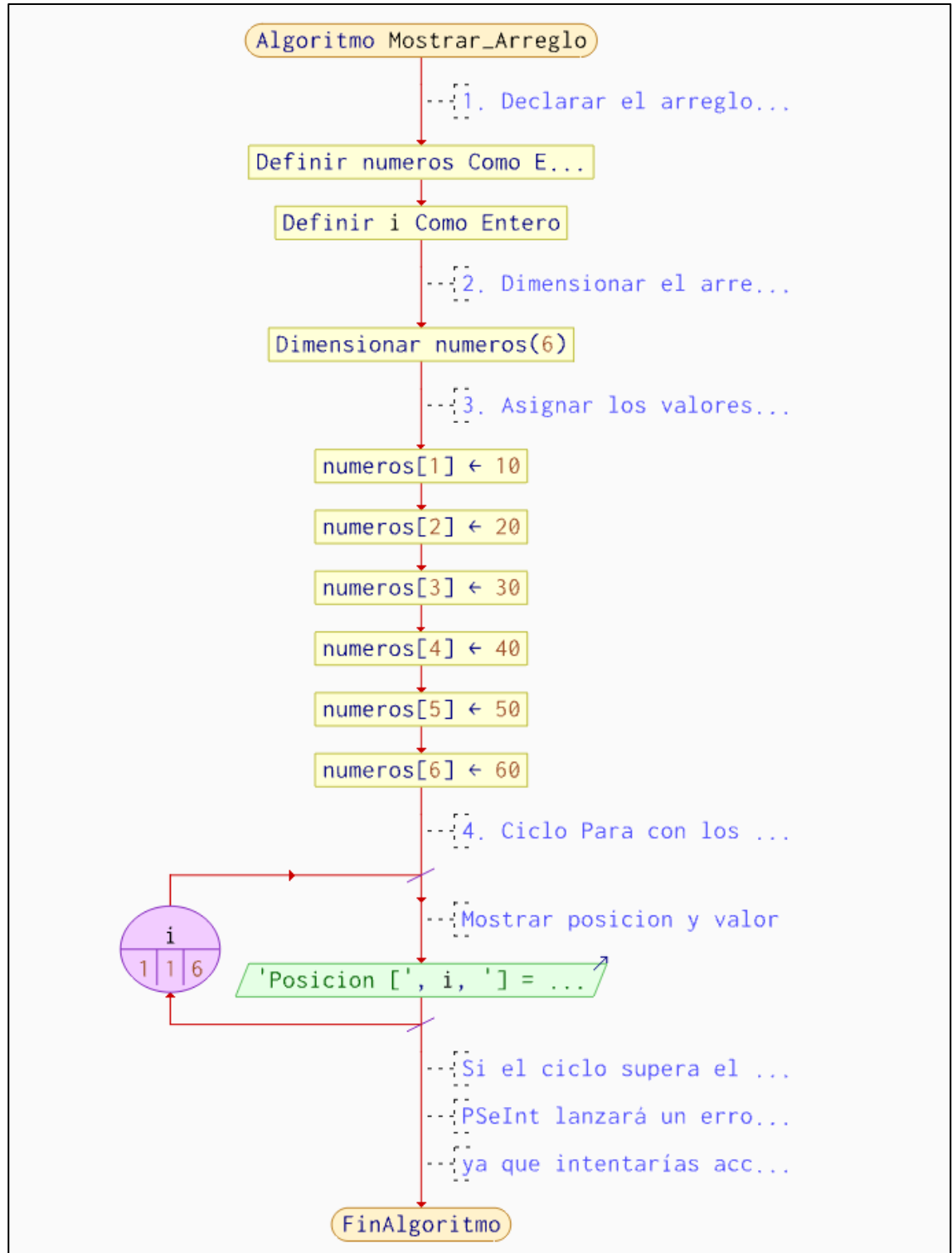


Ilustración 0.12:Diagrama de flujo

6) **Código en C++.**
Ejercicio 2 en C++

```
#include <iostream>

using namespace std;

int main() {

    // Arreglo de 6 números enteros
    int numeros[6] = {10, 20, 30, 40, 50, 60};

    // Ciclo for con limites correctos
    for (int i = 0; i < 6; i++) {

        //Mostrar posicion y valor
        cout << "Posicion [" << i << "] = " << numeros[i] << endl;
    }

    /*
    Si el ciclo supera el tamaño del arreglo
    (por ejemplo i <= 6 o i < 10),
    el programa intentaría acceder a posiciones
    de memoria que no pertenecen al arreglo.
    Esto puede provocar errores, resultados
    incorrectos o desbordamiento de memoria.
    */

    return 0;
}
```

7) Código en Java.
Ejercicio 2 en java

```
public class Ejercicio2 {

    public static void main(String[] args) {

        // Arreglo con 6 números enteros
        int[] numeros = {-8, 19, -3, 5, 13, 100};
```



```
// Ciclo for con límites correctos
for (int i = 0; i < numeros.length; i++) {

    // Mostrar posición y valor
    System.out.println("Posición " + i + " = " + numeros[i]);
}

/*
 * El arreglo tiene posiciones de 0 a 5.
 * Cuando i llega a 6, el programa intenta acceder
 * a una posición que no existe y genera el error:
 * ArrayIndexOutOfBoundsException
 */
}
```

8) Pruebas de solución.

```
#include <iostream>
using namespace std;

int main() {
    // Arreglo de 6 números enteros
    int numeros[6] = {10, 20, 30, 40, 50, 60};

    // Ciclo for con límites correctos
    for (int i = 0; i < 6; i++) {

        //Mostrar posición y valor
        cout << "Posición [" << i << "] = " << numeros[i] << endl;
    }

    /*
     Si el ciclo supera el tamaño del arreglo
     (por ejemplo i <= 6 o i < 10),
     el programa intentaría acceder a posiciones
     de memoria que no pertenecen al arreglo.
     Esto puede provocar errores, resultados
     incorrectos o desbordamiento de memoria.
    */

    return 0;
}
```

Ilustración 0.13: Código de C++ en Visual Studio Core

```
PS C:\Users\LENOVO LOQ\Documents\Visual Studio Code\C++> cd "c:\Users\LENOVO LOQ\Documents\Visual Studio Code\C++\" ; if ($?) { g++ Tarea6_2.cpp -o Tarea6_2 } ; if ($?) { .\Tarea6_2 }
Posición [0] = 10
Posición [1] = 20
Posición [2] = 30
Posición [3] = 40
Posición [4] = 50
Posición [5] = 60
PS C:\Users\LENOVO LOQ\Documents\Visual Studio Code\C++>
```

Ilustración 0.14: Prueba de solución 1 C++

```
PS C:\Users\LENOVO LOQ\Documents\Visual Studio Code\C++> cd "c:\Users\LENOVO LOQ\Documents\Visual Studio Code\C++\" ; if ($?) { g++ Tarea6_2.cpp -o Tarea6_2 } ; if ($?) { .\Tarea6_2 }
Posición [0] = -8
Posición [1] = 19
Posición [2] = -3
Posición [3] = 5
Posición [4] = 13
Posición [5] = 100
PS C:\Users\LENOVO LOQ\Documents\Visual Studio Code\C++>
```

Ilustración 0.15: Prueba de solución C++

```
Ejercicio2.java > Ejercicio2 > main(String[])
1 public class Ejercicio2 {
    Run | Debug
2     public static void main(String[] args) {
3
4         // Arreglo con 6 números enteros
5         int[] numeros = {-8, 19, -3, 5, 13, 100};
6
7         // Ciclo for con límites correctos
8         for (int i = 0; i < numeros.length; i++) {
9
10            // Mostrar posición y valor
11            System.out.println("Posición " + i + " = " + numeros[i]);
12        }
13
14        /*
15        * El arreglo tiene posiciones de 0 a 5.
16        * Cuando i llega a 6, el programa intenta acceder
17        * a una posición que no existe y genera el error:
18        * ArrayIndexOutOfBoundsException
19        */
20    }
21 }
22
23
```

Ilustración 0.16: Código de Java en Visual Studio Core

```
preview -xx:+showCodeDetailsInExceptionMessages -cp C:\Users\HP\AppData\Roaming\Code\cf5d5\redhat.java\jdt_ws\POO_47781a3\bin' 'Ejercicio2'
Posición 0 = -8
Posición 1 = 19
Posición 2 = -3
Posición 3 = 5
Posición 4 = 13
Posición 5 = 100
PS C:\Users\HP\Documents\GitHub\POO>
```

Ilustración 0.17: Prueba de solución 1 Java

```
Posición 0 = -8
Posición 1 = 19
Posición 2 = -3
Posición 3 = 5
Posición 4 = 13
Posición 5 = 100
Exception in thread "main" java.lang.ArrayIndexOutOfBoundsException: Index 6 out of bounds for length 6
    at Ejercicio2.main(Ejercicio2.java:11)
PS C:\Users\HP\Documents\GitHub\POO>
```

Ilustración 0.18: Prueba de solución 2 Java

CONCLUSIONES.

- Se implementó con éxito un sistema de búsqueda lineal sobre arreglos paralelos en Java, demostrando que la correspondencia de índices es un método efectivo para la gestión integrada de datos relacionados (como nombres y precios de productos), logrando además optimizar el proceso iterativo al detener el flujo inmediatamente tras el hallazgo del elemento objetivo mediante la instrucción break.
- Se comprobó que la correcta delimitación de las condiciones de parada en los ciclos iterativos ($i < \text{numeros.length}$) es un mecanismo fundamental de seguridad informática que evita el desbordamiento de memoria; asimismo, se identificó de forma práctica que superar el tamaño asignado a un arreglo invoca de forma perjudicial la excepción `java.lang.ArrayIndexOutOfBoundsException`, interrumpiendo el flujo normal del sistema.
- Las pruebas de ejecución ejecutadas en el entorno de desarrollo permitieron contrastar la diferencia entre un procesamiento de datos controlado y un fallo por desbordamiento, consolidando la importancia de la fase de testing para asegurar que las aplicaciones no solo resuelvan el problema matemático/lógico planteado, sino que mantengan su integridad operativa ante ingresos erróneos o búsquedas fallidas.

RECOMENDACIONES.

- **Utilizar estructuras de datos avanzadas para búsquedas extensas:** Si bien los arreglos paralelos y la búsqueda lineal resuelven de forma óptima el problema para conjuntos de datos reducidos, se recomienda migrar hacia Colecciones (como `HashMap` o `ArrayList`) en entornos de producción reales, lo que facilitará el mantenimiento del código y mejorará la complejidad temporal de las consultas en inventarios masivos.
- **Implementar bloques de control de excepciones (Try-Catch):** Para robustecer la prevención del desbordamiento de memoria y otros errores inesperados en tiempo de ejecución, se sugiere envolver los ciclos de lectura y manipulación de datos dentro de bloques try-catch, permitiendo capturar de manera elegante cualquier eventualidad del tipo `ArrayIndexOutOfBoundsException` sin que la aplicación sufra un cierre abrupto.
- **Modularizar el código fuente:** Se aconseja separar las responsabilidades de los programas implementando métodos específicos para la captura de datos, el procesamiento/búsqueda y la impresión de salidas en pantalla; esto no solo incrementará la legibilidad y la reutilización de los algoritmos de integridad, sino que también facilitará la documentación y el trabajo colaborativo dentro del equipo de desarrollo.

ANEXOS.

```
int main(){
    //Declaracion de productos y precios
    string producto[5]={"Leche", "Pan", "Yogurt", "Cereal", "Sandia"};
    float precio[5]={1,0.25,0.75,1.5,2.75};

    //Declaracion de variables a buscar
    string buscar;
    bool encontrado = false;

    //Pedir el producto al usuario
    cout << "Ingrese el producto que desea buscar: ";
    cin >> buscar;

    //Busqueda lineal con un for
    for(int i=0; i<5; i++){

        if(producto[i]==buscar){
            cout << "Producto encontrado!\n";
            cout << "Producto: " << producto[i]
            << "\nPrecio: " << precio[i] << endl;

            encontrado=true;
            break;
        }
    }

    //En caso de que no se encuentre el producto
    if(!encontrado){
        cout << "No se dispone del producto ingresado";
    }
    return 0;
}
```

Ilustración 0.1: Código de C++ en Visual Studio Core

```
PS C:\Users\LENOVO LOQ\Documents\Visual Studio Code\C++> cd "c:\Users\LENOVO LOQ\Documents\Visual Studio
Code\C++\" ; if ($?) { g++ Tarea6.cpp -o Tarea6 } ; if ($?) { .\Tarea6 }
Ingrese el producto que desea buscar: Leche
Producto encontrado!
Producto: Leche
Precio: 1
```

Ilustración 0.2: Prueba de solución 1 C++

```
PS C:\Users\LENOVO LOQ\Documents\Visual Studio Code\C++> cd "c:\Users\LENOVO LOQ\Documents\Visual Studio
Code\C++\" ; if ($?) { g++ Tarea6.cpp -o Tarea6 } ; if ($?) { .\Tarea6 }
Ingrese el producto que desea buscar: Pan
Producto encontrado!
Producto: Pan
Precio: 0.25
PS C:\Users\LENOVO LOQ\Documents\Visual Studio Code\C++> █
```

Ilustración 0.3: Prueba de solución 2 C++

```
PS C:\Users\LENOVO LOQ\Documents\Visual Studio Code\C++> cd "c:\Users\LENOVO LOQ\Documents\Visual Studio
Code\C++\" ; if ($?) { g++ Tarea6.cpp -o Tarea6 } ; if ($?) { .\Tarea6 }
Ingrese el producto que desea buscar: Jamon
No se dispone del producto ingresado
PS C:\Users\LENOVO LOQ\Documents\Visual Studio Code\C++> █
```

Ilustración 0.4: Prueba de solución 3 C++

```
PS C:\Users\LENOVO LOQ\Documents\Visual Studio Code\C++> cd "c:\Users\LENOVO LOQ\Documents\Visual Studio Code\C++\" ; if ($?) { g++ Tarea6.cpp -o Tarea6 } ; if ($?) { .\Tarea6 }
Ingrese el producto que desea buscar: Sandia
Producto encontrado!
Producto: Sandia
Precio: 2.75
PS C:\Users\LENOVO LOQ\Documents\Visual Studio Code\C++> |
```

Ilustración 0.5: Prueba de solución 4 C++

```
J BuscarProducto.java > BuscarProducto > main(String[])
1 import java.util.Scanner;
2
3 public class BuscarProducto {
4     public static void main(String[] args) {
5         // Declaración de un arreglo de productos
6         String[] producto = {"Leche", "Pan", "Yogurt", "Cereal", "Sandia"};
7         // Declaración de un arreglo de precios (correspondiente a cada producto)
8         float[] precio = {1f, 0.25f, 0.75f, 1.5f, 2.75f};
9
10        // Objeto Scanner para leer datos desde teclado
11        Scanner sc = new Scanner(System.in);
12
13        // Solicitar al usuario el producto que desea buscar
14        System.out.print(s: "Ingrese el producto que desea buscar: ");
15        String buscar = sc.nextLine();
16
17        // Variable booleana para indicar si se encontró el producto
18        boolean encontrado = false;
19
20        // Recorrer el arreglo de productos
21        for (int i = 0; i < producto.length; i++) {
22            // Comparar el producto ingresado con el arreglo (ignora mayúsculas/minúsculas)
23            if (producto[i].equalsIgnoreCase(buscar)) {
24                System.out.println(x: "Producto encontrado!");
25                System.out.println("Producto: " + producto[i]);
26                System.out.println("Precio: " + precio[i]);
27                encontrado = true; // marcar que sí se encontró
28                break; // salir del ciclo porque ya se encontró
29            }
30        }
31
32        // Si no se encontró el producto, mostrar mensaje
33        if (!encontrado) {
34            System.out.println(x: "No se dispone del producto ingresado");
35        }
36
37        // Cerrar el objeto Scanner
38        sc.close();
39    }
40 }
```

Ilustración 0.6: Código de Java en Visual Studio Core

```
PS C:\Users\HP\Documents\GitHub\P00> c:.; cd 'c:\Users\HP\Documents\GitHub\P00';
onMessages' '-cp' 'C:\Users\HP\AppData\Roaming\Code\User\workspaceStorage\d16aa3
Ingrese el producto que desea buscar: Leche
Producto encontrado!
Producto: Leche
Precio: 1.0
```

Ilustración 0.7: Prueba de solución 1 Java

```
S C:\Users\HP\Documents\GitHub\P00> c:.; cd 'c:\Users\HP\Documents\GitHub\P00';
onMessages' '-cp' 'C:\Users\HP\AppData\Roaming\Code\User\workspaceStorage\d16aa3
ngrese el producto que desea buscar: queso
o se dispone del producto ingresado
S C:\Users\HP\Documents\GitHub\P00> |
```

Ilustración 0.8: Prueba de solución 2 Java

```
PS C:\Users\HP\Documents\GitHub\P00> c::; cd 'c:\Users\HP\Documents\GitHub\P00';
onMessages' '-cp' 'C:\Users\HP\AppData\Roaming\Code\User\workspaceStorage\d16aa:
Ingrese el producto que desea buscar: Cereal
Producto encontrado!
Producto: Cereal
Precio: 1.5
PS C:\Users\HP\Documents\GitHub\P00> █
```

Ilustración 0.9: Prueba de solución 3 Java

```
#include <iostream>
using namespace std;

int main() {
    // Arreglo de 6 números enteros
    int numeros[6] = {10, 20, 30, 40, 50, 60};

    // Ciclo for con limites correctos
    for (int i = 0; i < 6; i++) {

        //Mostrar posicion y valor
        cout << "Posicion [" << i << "] = " << numeros[i] << endl;
    }

    /*
    Si el ciclo supera el tamaño del arreglo
    (por ejemplo i <= 6 o i < 10),
    el programa intentaría acceder a posiciones
    de memoria que no pertenecen al arreglo.
    Esto puede provocar errores, resultados
    incorrectos o desbordamiento de memoria.
    */

    return 0;
}
```

Ilustración 0.10: Código de C++ en Visual Studio Core

```
PS C:\Users\LENOVO LOQ\Documents\Visual Studio Code\C++> cd "c:\Users\LENOVO LOQ\Documents\Visual Studio
Code\C++\" ; if ($?) { g++ Tarea6_2.cpp -o Tarea6_2 } ; if ($?) { .\Tarea6_2 }
Posicion [0] = 10
Posicion [1] = 20
Posicion [2] = 30
Posicion [3] = 40
Posicion [4] = 50
Posicion [5] = 60
PS C:\Users\LENOVO LOQ\Documents\Visual Studio Code\C++>
```

Ilustración 0.11: Prueba de solución 1 C++

```
PS C:\Users\LENOVO LOQ\Documents\Visual Studio Code\C++> cd "c:\Users\LENOVO LOQ\Documents\Visual Studio
Code\C++\" ; if ($?) { g++ Tarea6_2.cpp -o Tarea6_2 } ; if ($?) { .\Tarea6_2 }
Posicion [0] = -8
Posicion [1] = 19
Posicion [2] = -3
Posicion [3] = 5
Posicion [4] = 13
Posicion [5] = 100
PS C:\Users\LENOVO LOQ\Documents\Visual Studio Code\C++>
```

Ilustración 0.12: Prueba de solución C++

```

J Ejercicio2.java > Ejercicio2 > main(String[])
1 public class Ejercicio2 {
2     Run | Debug
3     public static void main(String[] args) {
4         // Arreglo con 6 números enteros
5         int[] numeros = {-8, 19, -3, 5, 13, 100};
6
7         // Ciclo for con límites correctos
8         for (int i = 0; i < numeros.length; i++) {
9
10            // Mostrar posición y valor
11            System.out.println("Posición " + i + " = " + numeros[i]);
12        }
13
14        /*
15        * El arreglo tiene posiciones de 0 a 5.
16        * Cuando i llega a 6, el programa intenta acceder
17        * a una posición que no existe y genera el error:
18        * ArrayIndexOutOfBoundsException
19        */
20    }
21 }
22
23

```

Ilustración 0.13: Código de Java en Visual Studio Code

```

preview -xx:+showCodeDetailsInExceptionMessages -cp C:\Users\HP\AppData\Roaming\Code\
cfd5\redhat.java\jdt_ws\P00_47781a3\bin' 'Ejercicio2'
Posición 0 = -8
Posición 1 = 19
Posición 2 = -3
Posición 3 = 5
Posición 4 = 13
Posición 5 = 100
PS C:\Users\HP\Documents\GitHub\P00>

```

Ilustración 0.14: Prueba de solución 1 Java

```

Posición 0 = -8
Posición 1 = 19
Posición 2 = -3
Posición 3 = 5
Posición 4 = 13
Posición 5 = 100
Exception in thread "main" java.lang.ArrayIndexOutOfBoundsException: Index 6 out of bounds for length 6
    at Ejercicio2.main(Ejercicio2.java:11)
PS C:\Users\HP\Documents\GitHub\P00>

```

Ilustración 0.15: Prueba de solución 2 Java